

Lab 11 – Data Structures with AI: Implementing Fundamental Structures

Name: N.rahul

Roll_no: 2503A52403

Batch No: 14

Task Description :1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Input:

```
class Stack:
```

```
    """A simple Stack implementation using a list."""
```

```
    def __init__(self):
```

```
        """Initialize an empty stack."""
```

```
        self.items = []
```

```
    def push(self, item):
```

```
        """Add an item to the top of the stack."""
```

```
self.items.append(item)
```

```
def pop(self):
```

```
    """Remove and return the top item from the  
    stack."""
```

```
    if self.is_empty():
```

```
        return None
```

```
    return self.items.pop()
```

```
def peek(self):
```

```
    """Return the top item without removing it.""" if
```

```
self.is_empty():
```

```
    return None return
```

```
self.items[-1]
```

```
def is_empty(self):
```

```
    """Return True if the stack is empty, otherwise  
    False."""
```

```
    return len(self.items) == 0
```

```
# Example usage
```

```
stack = Stack()
```

```
stack.push(10)
```

```
stack.push(20)
```

```
stack.push(30)
```

```
print("Top element:", stack.peek())
```

```
print("Popped element:", stack.pop())
```

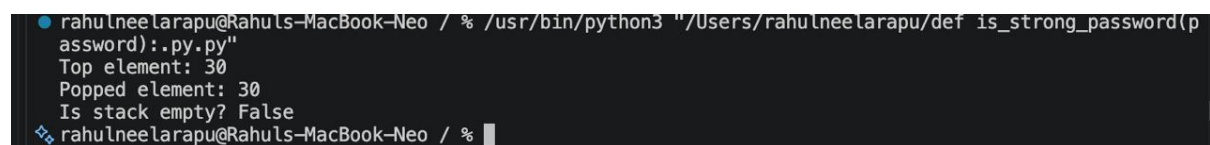
```
print("Is stack empty?", stack.is_empty())
```

Explanation:

A **Stack** is a LIFO (Last In, First Out) data structure where the last inserted element is removed first.

push() adds an element, pop() removes the top element, and peek() returns the top element without removing it. is_empty() checks whether the stack contains any elements.

Output:



```
● rahulneelarapu@Rahuls-MacBook-Neo / % /usr/bin/python3 "/Users/rahulneelarapu/def is_strong_password(p
assword):.py.py"
Top element: 30
Popped element: 30
Is stack empty? False
❖ rahulneelarapu@Rahuls-MacBook-Neo / %
```

Task Description :2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Input:

```
class Queue:
```

```
    """A simple Queue implementation using a Python list."""
```

```
    def __init__(self):
```

```
        """Initialize an empty queue."""
```

```
        self.items = []
```

```
    def enqueue(self, item):
```

```
        """Add an item to the rear of the queue."""
```

```
        self.items.append(item)
```

```
    def dequeue(self):
```

```
        """Remove and return the front item from the queue."""
```

```
        if len(self.items) == 0: return
```

```
        None
```

```
        return self.items.pop(0)
```

```
    def peek(self):
```

```
        """Return the front item without removing it."""
```

```
if len(self.items) == 0: return  
    None  
return self.items[0]
```

```
def size(self):  
    """Return the number of items in the queue."""  
    return len(self.items)
```

Example usage

```
queue = Queue()
```

```
queue.enqueue(10)
```

```
queue.enqueue(20)
```

```
queue.enqueue(30)
```

```
print("Front element:", queue.peek()) print("Dequeued  
element:", queue.dequeue()) print("Queue size:",  
queue.size())
```

Explanation:

A **Queue** is a FIFO (First In, First Out) data structure, where the first inserted element is removed first.

enqueue() adds an element, dequeue() removes the front element, and peek() returns the front element without removing it.
size() returns the number of elements currently present in the queue.

Output:

```
● rahulneelarapu@Rahuls-MacBook-Neo / % /usr/bin/python3 "/Users/rahulneelarapu/def is_strong_password(p
assword):.py.py"
Front element: 10
Dequeued element: 10
Queue size: 2
❖ rahulneelarapu@Rahuls-MacBook-Neo / % █
```

Task Description :3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Input:

class Node:

"""Represents a node in a singly linked list."""

def __init__(self, data):

"""Initialize a node with data and a next pointer."""

self.data = data

self.next = None

```
class LinkedList:
```

```
    """A simple singly linked list implementation."""
```

```
    def __init__(self):
```

```
        """Initialize an empty linked list."""
```

```
        self.head = None
```

```
    def insert(self, data):
```

```
        """Insert a new node at the end of the linked list."""
```

```
        new_node = Node(data)
```

```
        if self.head is None:
```

```
            self.head = new_node
```

```
            return
```

```
        current = self.head
```

```
        while current.next is not None: current =
```

```
            current.next
```

```
        current.next = new_node
```

```
    def display(self):
```

```
        """Display all elements of the linked list."""
```

```
current = self.head
```

```
while current is not None:
```

```
    print(current.data, end=" -> ") current  
    = current.next
```

```
print("None")
```

```
# Example usage
```

```
linked_list = LinkedList()
```

```
linked_list.insert(10)
```

```
linked_list.insert(20)
```

```
linked_list.insert(30)
```

```
linked_list.insert(40)
```

```
print("Linked List:")
```

```
linked_list.display()
```

Explanation:

A **Singly Linked List** is a linear data structure made of nodes, where each node stores data and a link to the next node.

The insert() method adds a new node to the list, while the display() method prints all elements from the first node to the last.

Each node points only to the next node, and the last node points to None.

Output:

```
● rahulneelarapu@Rahuls-MacBook-Neo / % /usr/bin/python3 "/Users/rahulneelarapu/def is_strong_password(p
assword):.py.py"
10 -> 20 -> 30 -> 40 -> None
rahulneelarapu@Rahuls-MacBook-Neo / %
```

Task Description :4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

Input:

class Node:

"""Represents a node in a Binary Search Tree."""

def __init__(self, data):

"""Initialize a node with data and two child pointers."""

self.data = data

self.left = None

```
self.right = None
```

```
class BST:
```

```
    """Binary Search Tree with recursive insertion and  
    traversal."""
```

```
    def __init__(self):
```

```
        """Initialize an empty Binary Search Tree."""
```

```
        self.root = None
```

```
    def insert(self, root, data):
```

```
        """Recursively insert a value into the BST.""" if
```

```
        root is None:
```

```
            return Node(data)
```

```
        if data < root.data:
```

```
            root.left = self.insert(root.left, data)
```

```
        elif data > root.data:
```

```
            root.right = self.insert(root.right, data) return
```

```
        root
```

```
def inorder(self, root):  
    """Perform recursive in-order traversal of the  
    BST."""  
    if root is not None: self.inorder(root.left)  
        print(root.data, end=" ")  
        self.inorder(root.right)
```

```
# Example usage bst  
= BST()
```

```
values = [50, 30, 70, 20, 40, 60, 80]
```

```
for value in values:  
    bst.root = bst.insert(bst.root, value)
```

```
print("In-order Traversal:")
```

```
bst.inorder(bst.root)
```

Explanation:

A Binary Search Tree (BST) is a tree where smaller values are stored on the left and larger values on the right. The insert() method recursively places a new value in its

correct position in the tree.

The inorder() traversal recursively visits **left** → **root** → **right**, producing values in sorted order.

Output:

```
rahulneelarapu@Rahuls-MacBook-Neo / % /usr/bin/python3 "/Users/rahulneelarapu/def is_strong_password(p
assword):.py.py"
In-order traversal:
20 30 40 50 60 70 80
rahulneelarapu@Rahuls-MacBook-Neo / %
```

Task Description :5 – Priority Queue

Task: Use AI to implement a priority queue using Python's heapq

Input:

```
import heapq
```

```
class PriorityQueue:
```

```
    """Priority Queue implementation using Python's
    heapq module."""
```

```
    def __init__(self):
```

```
        """Initialize an empty priority queue."""
```

```
        self.queue = []
```

```
    def enqueue(self, priority, item):
```

```
        """Add an item with the given priority to the
queue."""
```

```
        heapq.heappush(self.queue, (priority, item))
```

```
def dequeue(self):
```

```
    """Remove and return the item with the highest
priority."""
```

```
    if not self.queue:
```

```
        return None
```

```
    priority, item = heapq.heappop(self.queue) return
    item
```

```
def display(self):
```

```
    """Display all items in the priority queue."""
```

```
    print("Priority Queue:")
```

```
    for priority, item in sorted(self.queue):
```

```
        print(f"Priority {priority}: {item}")
```

```
# Example usage pq =
```

```
PriorityQueue()
```

```
pq.enqueue(3, "Task C")
pq.enqueue(1, "Task A")
pq.enqueue(2, "Task B")

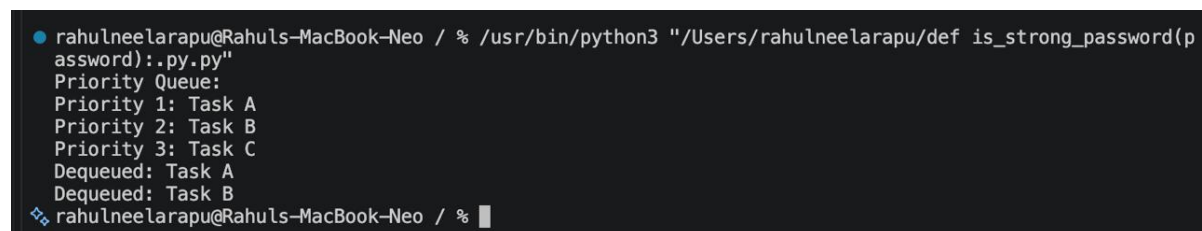
pq.display()

print("Dequeued:", pq.dequeue())
print("Dequeued:", pq.dequeue())
```

Explanation:

A **Priority Queue** is a data structure where each element is served according to its priority instead of insertion order. Python's `heapq` module implements a **min-heap**, so a smaller priority number represents a higher priority in this example.

Output:



```
● rahulneelarapu@Rahuls-MacBook-Neo / % /usr/bin/python3 "/Users/rahulneelarapu/def is_strong_password(p
assword):.py.py"
Priority Queue:
Priority 1: Task A
Priority 2: Task B
Priority 3: Task C
Dequeued: Task A
Dequeued: Task B
❖ rahulneelarapu@Rahuls-MacBook-Neo / % █
```

Task Description :6 – Deque

Task: Use AI to implement a double-ended queue using `collections.deque`.

Input:

```
from collections import deque
```

```
class DequeDS:
```

```
    """A double-ended queue implementation using  
collections.deque."""
```

```
    def __init__(self):
```

```
        """Initialize an empty deque."""
```

```
        self.items = deque()
```

```
    def insert_front(self, item):
```

```
        """Insert an item at the front of the deque."""
```

```
        self.items.appendleft(item)
```

```
    def insert_rear(self, item):
```

```
        """Insert an item at the rear of the deque."""
```

```
        self.items.append(item)
```

```
    def remove_front(self):
```

```
        """Remove and return the item from the front.""" if  
        not self.items:
```

```
            return None
```

```
        return self.items.popleft()
```

```
def remove_rear(self):
```

```
    """Remove and return the item from the rear.""" if
```

```
    not self.items:
```

```
        return None
```

```
    return self.items.pop()
```

```
def display(self):
```

```
    """Display all elements in the deque.""" print("Deque:",
```

```
    list(self.items))
```

```
# Example usage
```

```
dq = DequeDS()
```

```
dq.insert_rear(10)
```

```
dq.insert_rear(20)
```

```
dq.insert_front(5)
```

```
dq.display()
```

```
print("Removed from front:", dq.remove_front())
```

```
print("Removed from rear:", dq.remove_rear())
```

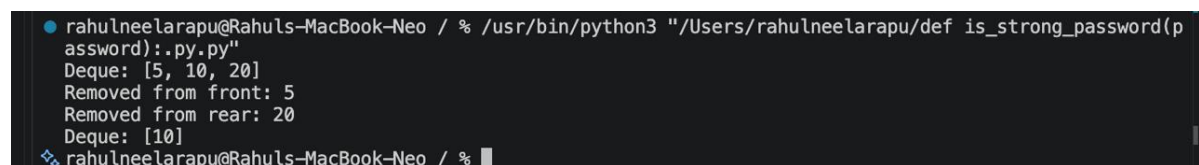

`dq.display()`

Explanation:

A **Deque (Double-Ended Queue)** allows insertion and deletion of elements from both the front and rear ends.

Python's `collections.deque` provides efficient methods such as `append()`, `appendleft()`, `pop()`, and `popleft()`.

Output:

A terminal window screenshot showing the execution of a Python script. The prompt is 'rahulneelarapu@Rahuls-MacBook-Neo / %'. The command is '/usr/bin/python3 "/Users/rahulneelarapu/def is_strong_password(p'. The output shows a deque with elements [5, 10, 20], followed by 'Removed from front: 5', 'Removed from rear: 20', and the final deque state [10]. The prompt then changes to 'rahulneelarapu@Rahuls-MacBook-Neo / %' with a cursor.

```
rahulneelarapu@Rahuls-MacBook-Neo / % /usr/bin/python3 "/Users/rahulneelarapu/def is_strong_password(p
assword):.py.py"
Deque: [5, 10, 20]
Removed from front: 5
Removed from rear: 20
Deque: [10]
rahulneelarapu@Rahuls-MacBook-Neo / %
```

Task Description :7 Real-Time Application Challenge –
Choose the

Right Data Structure

Scenario:

Your college wants to develop a Campus Resource
Management System

that handles:

1. Student Attendance Tracking – Daily log of students entering/exiting the campus.
2. Event Registration System – Manage participants in events with quick search and removal.

3. Library Book Borrowing – Keep track of available books and their due dates.

4. Bus Scheduling System – Maintain bus routes and stop connections.

5. Cafeteria Order Queue – Serve students in the order they arrive.

Input:

```
from collections import deque
```

```
class CafeteriaQueue:
```

```
    """Manage cafeteria orders using a FIFO queue."""
```

```
    def __init__(self):
```

```
        """Initialize an empty cafeteria order queue."""
```

```
        self.orders = deque()
```

```
    def add_order(self, student_name, order):
```

```
        """Add a student's order to the rear of the queue."""
```

```
        self.orders.append((student_name, order))
```

```
def serve_order(self):  
    """Remove and serve the order at the front of the  
queue."""  
    if not self.orders:  
        return None  
  
    return self.orders.popleft()
```

```
def display_orders(self):  
    """Display all waiting cafeteria orders.""" if  
not self.orders:  
    print("No pending orders.") return  
  
    print("Pending Orders:")  
    for student, order in self.orders: print(f"{student}  
        - {order}")
```

```
# Example usage  
cafeteria = CafeteriaQueue()
```

```
# Students place their orders
cafeteria.add_order("Rahul", "Dosa")
cafeteria.add_order("Priya", "Burger")
cafeteria.add_order("Arjun", "Biryani")

# Display pending orders
cafeteria.display_orders()

# Serve orders in FIFO order
print("\nServing:", cafeteria.serve_order())
print("Serving:", cafeteria.serve_order())

# Display remaining orders
print("\nRemaining Orders:")
cafeteria.display_orders()
```

Explanation:

The **Queue** is appropriate for the cafeteria because it maintains the order in which students arrive. The implementation uses Python's deque to efficiently add orders at the rear and remove served orders from the front.

Output:

```
rahulneelarapu@Rahuls-MacBook-Neo / % /usr/bin/python3 "/Users/rahulneelarapu/def is_strong_password(p
assword):.py.py"
Pending Orders:
Rahul - Dosa
Priya - Burger
Arjun - Biryani
Serving: ('Rahul', 'Dosa')
Serving: ('Priya', 'Burger')
Pending Orders:
Arjun - Biryani
rahulneelarapu@Rahuls-MacBook-Neo / % █
```

Task Description :8: Smart E-Commerce Platform – Data Structure

Challenge

An e-commerce company wants to build a Smart Online Shopping System

with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.

Input:

```
from collections import deque
```

```
class OrderQueue:
```

```
    """Manage e-commerce orders using a FIFO
    queue."""
```

```
    def __init__(self):
```

```
        """Initialize an empty order queue.""" self.orders
        = deque()
```

```
    def add_order(self, order_id, customer_name): """Add
        a new order to the rear of the queue."""
```

```
        self.orders.append((order_id, customer_name))
```

```
    def process_order(self):
```

```
        """Remove and process the oldest order.""" if
        not self.orders:
```

```
            return None
```

```
        return self.orders.popleft()
```

```
def display_orders(self): """Display
    all pending orders.""" if not
    self.orders:
        print("No pending orders.") return

    print("Pending Orders:")
    for order_id, customer in self.orders:
        print(f"Order ID: {order_id}, Customer:
{customer}")
```

```
# Example usage order_queue =
OrderQueue()
```

```
# Add orders order_queue.add_order(101,
"Rahul") order_queue.add_order(102,
"Priya") order_queue.add_order(103,
"Arjun")
```

```
# Display pending orders
order_queue.display_orders()
```

```
# Process orders

print("\nProcessing:", order_queue.process_order())

print("Processing:", order_queue.process_order())


# Display remaining orders

print("\nRemaining Orders:")

order_queue.display_orders()
```

Explanation:

The **Queue** is suitable for order processing because it maintains the order in which customers place their orders. The `add_order()` method adds new orders, while `process_order()` removes the oldest order first.

Output:

```
assword):.py.py"
Pending Orders:
Order ID: 101, Customer: Rahul
Order ID: 102, Customer: Priya
Order ID: 103, Customer: Arjun
Processing: (101, 'Rahul')
Processing: (102, 'Priya')
Pending Orders:
Order ID: 103, Customer: Arjun
```